

Large Language Models (LLMs)

An introduction to LLMs and Natural Language Processing

Module Leader: Dr. Alessandro Di Stefano

Email: A.DiStefano@tees.ac.uk

School of Computing, Engineering & Digital Technologies

Outline

- Natural Language Processing and Language Modeling
- How do LLMs work?
- Recent LLMs

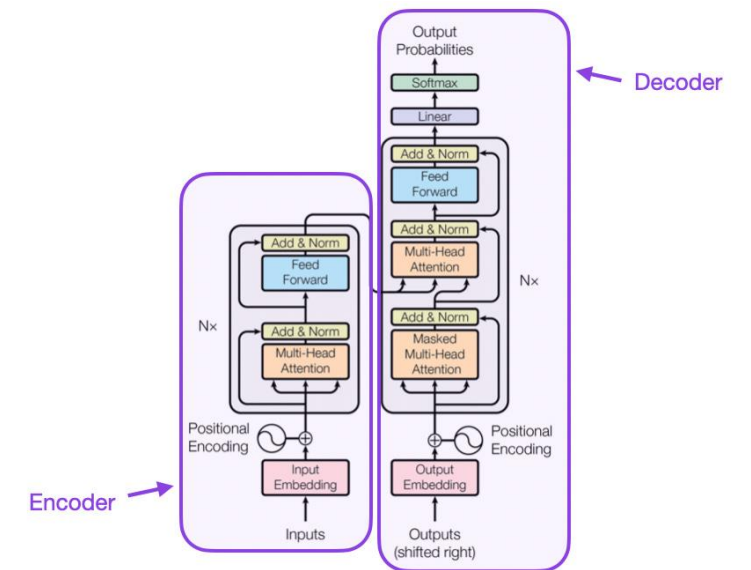


Figure 1: The Transformer - model architecture.

Natural Language Processing & Language Modeling

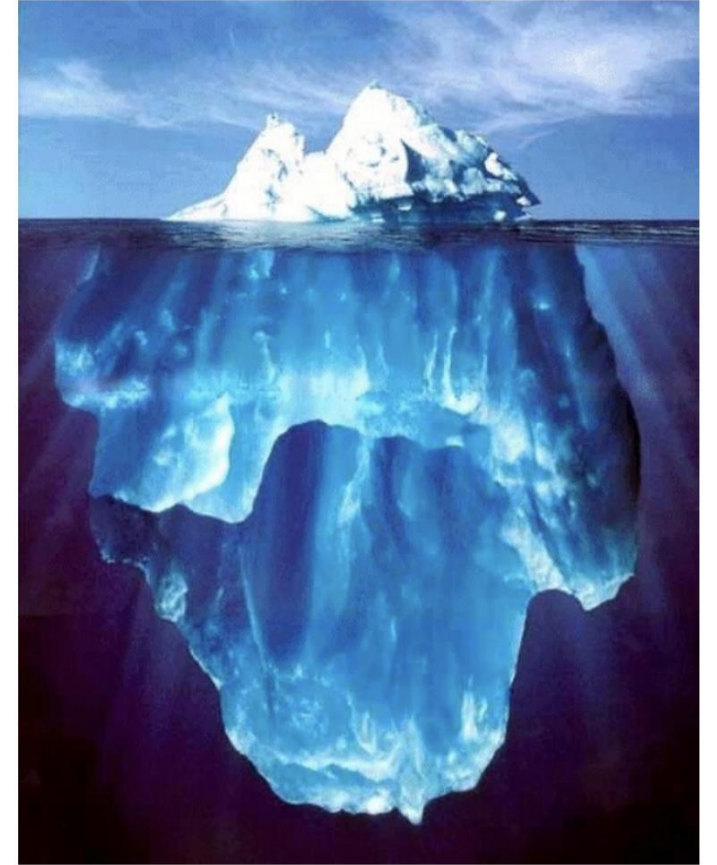
What is Natural Language Processing?

Natural language processing is a field at the intersection of Computer science, Artificial intelligence, Linguistics.

Goal: For computers to process or “understand” natural language in order to perform tasks that are useful, e.g., text summarization, text generation, language translation, etc.

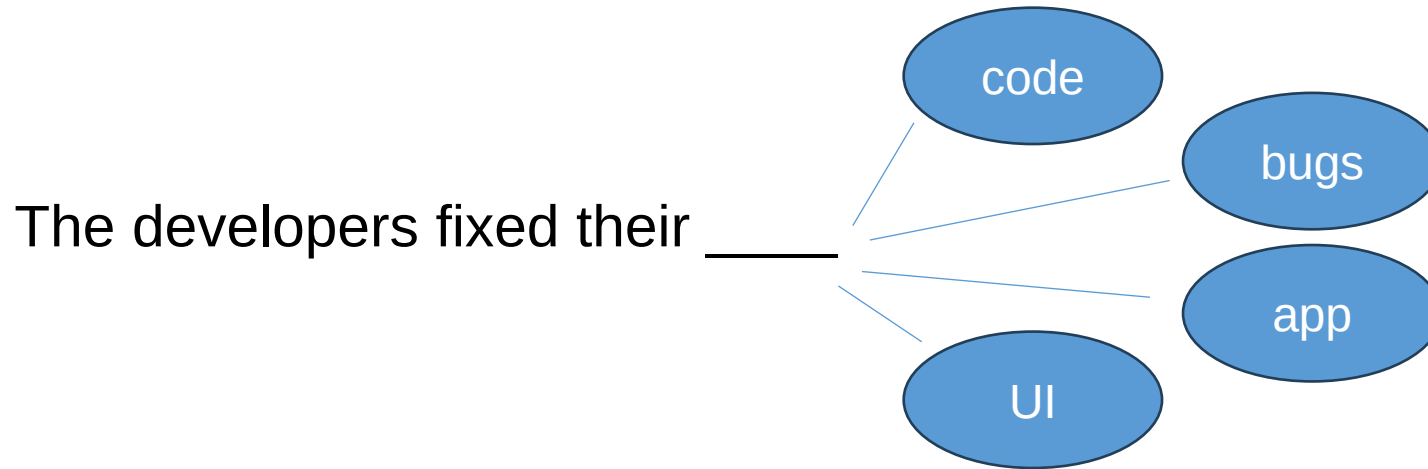
Why is it hard? Human languages are ambiguous and complex (unlike programming and other formal languages), e.g. "I made her duck." (ambiguity)

"An iceberg is a large piece of freshwater ice that has broken off from a snow-formed glacier or ice shelf and is floating in open water. “ (complexity)



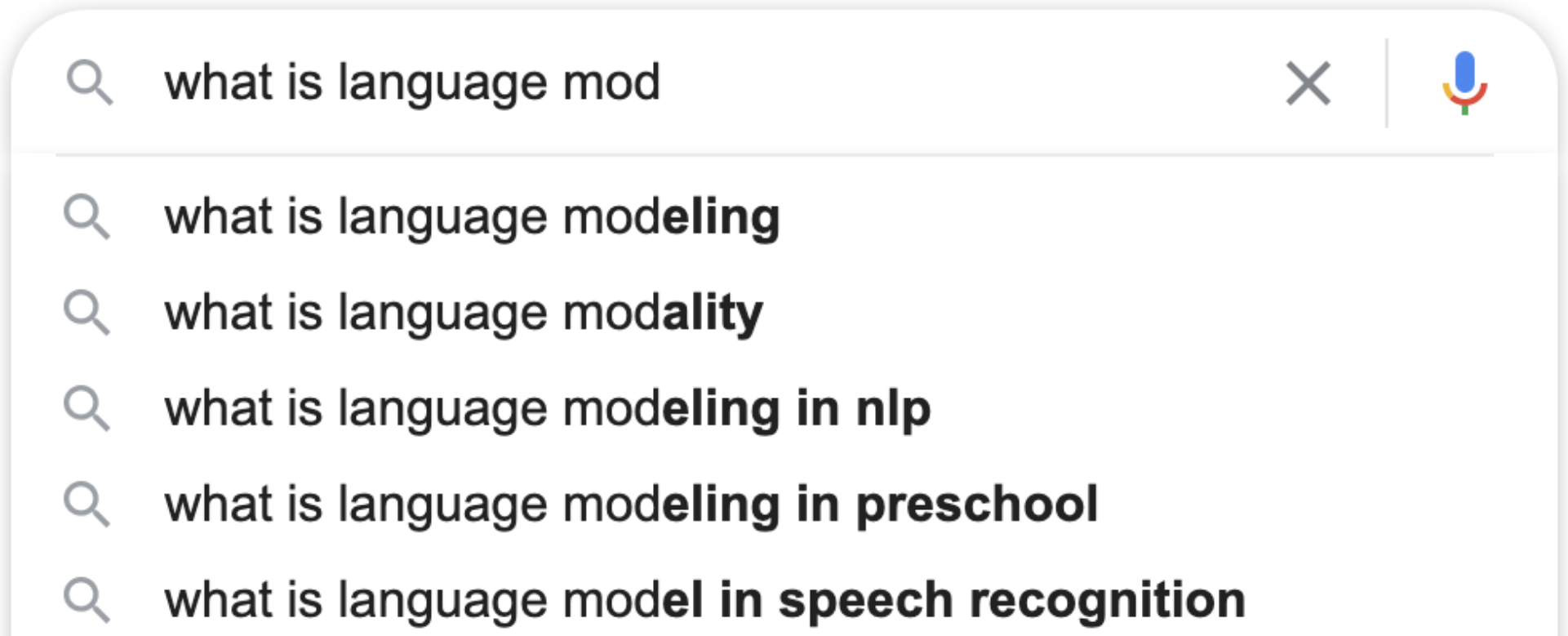
Language Modeling – Autoregressive learning

- **Language modeling** is the task of predicting what word comes next conditioning of words generated so far (Autoregressive learning)

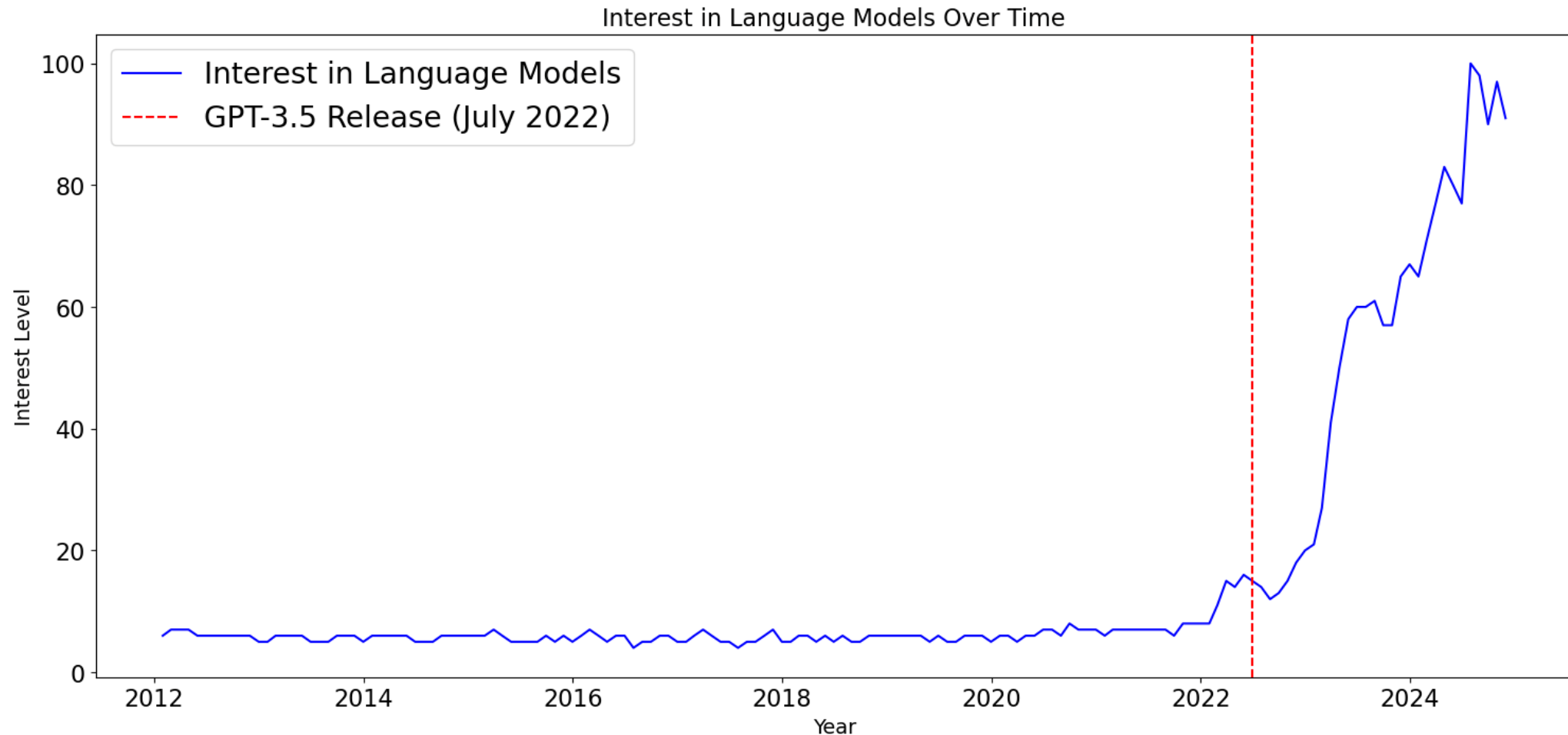


- An **autoregressive** model predicts the probability of next word x_t based on previous words $x_1, \dots, x_{t-1}$ in a sequence.
- The probability is denoted as $P(x_t | x_1, x_2 \dots x_{t-1})$ where P represents the conditional probability distribution of the next word based on the preceding ones.

You use language models everyday



Google trends for language models



Background: The Evolution of Word Encoding in NLP

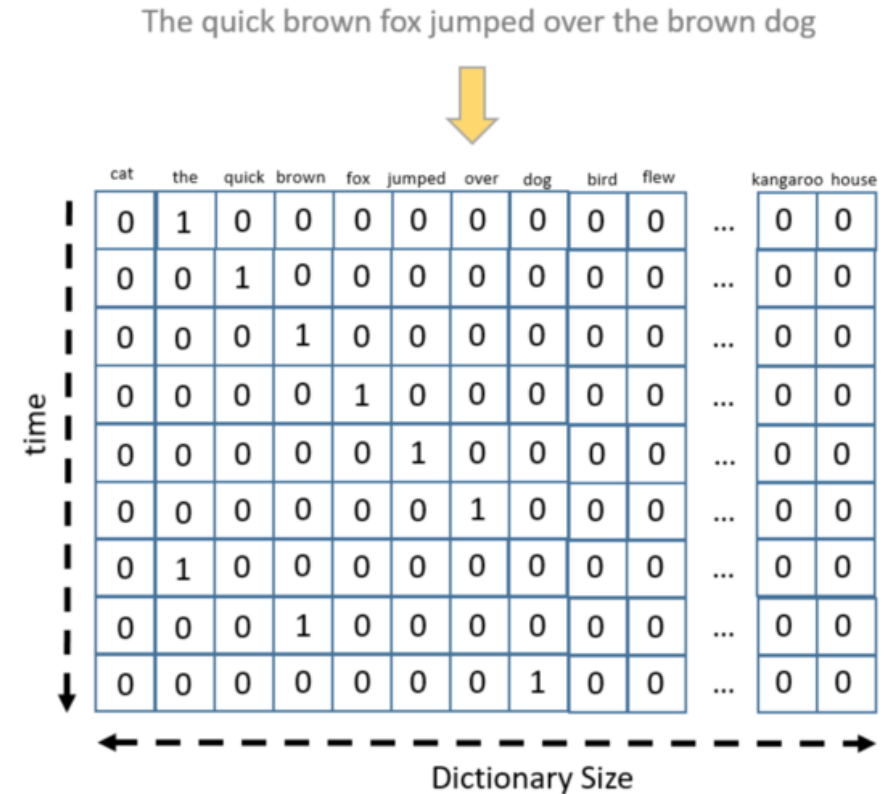
- Everything in NLP has now been rebuilt upon Language Modeling
- *"You shall know a word by the company it keeps"* - J.R. Firth, 1957
- Machine learning models operate in a mathematical, continuous space. They require numerical input to perform optimization, meaning words must be converted into a form that represents them as continuous values.
- **Lack of Uniformity:** Words are symbolic and arbitrary; 'Dog' and 'Canine' refer to the same animal but are completely different strings.
- Earlier in Machine learning course, we have used **One Hot Encoding** (OHE) to convert discrete values into vectors.

One Hot Encoding

- Process of converting words into a binary vector where each word is represented by a unique vector.
- A '1' indicates the presence of the word in the corresponding position, while '0's indicate its absence.

Problems

- **Sparsity:** The vectors are mostly zeros.
- **Scalability:** As the vocabulary grows, the vector dimensions increase linearly.
- **Lack of Context:** Does not capture any contextual or semantic relationships between words



From One-Hot Encoding to Word Vectors

- One-Hot Encoding cannot learn or infer the **semantic** connections between words, which are crucial for understanding language dynamics and subtleties.

A better Approach? Learn *Word Embeddings*!

- Embeddings** are dense, low-dimensional representations of words in a continuous vector space where similar words (meaning or functions) have similar encoding.
- Key Advantage:** we can use vector operations to model linguistic transformations between words.

King - Man + Woman = Queen

- Embeddings enable the modeling of relationships in high-dimensional space, allowing us to perform vector, e.g., transforming 'King' to 'Queen'.

Word Vectors: How does *Learning* happen?

- NLP algorithms learn **semantic relationships** between words from vast text of data (*word2Vec*)
- Adjust vectors to minimize prediction errors

Neural Word Vectors

- Represent words as high-dimensional vectors
- Similar words have similar vectors

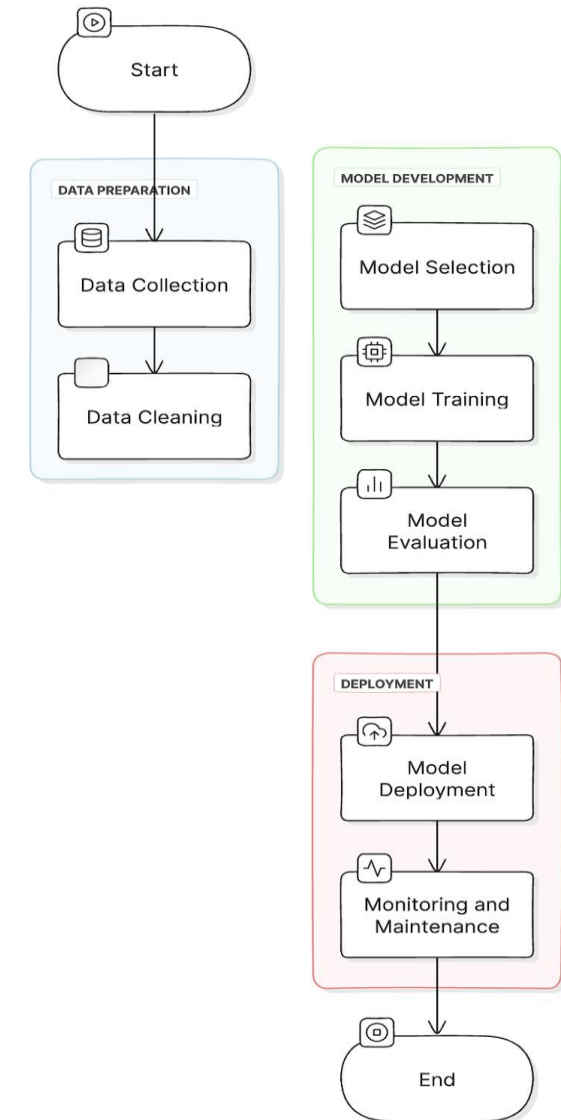
expect =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$


Mikolov, T., 2013. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

NLP learning Cycle

- Pick a problem (usually some disambiguation)
- Get a lot of data (usually a labeled corpus)
- Build the simplest thing that could possibly work
- **Repeat:**
 - Examine the most common errors
 - Figure out what information a human might use to avoid them
 - Modify the system to exploit that information:
 - Feature engineering
 - Representation redesign
 - Different machine learning methods
- We're doing this over and over again.



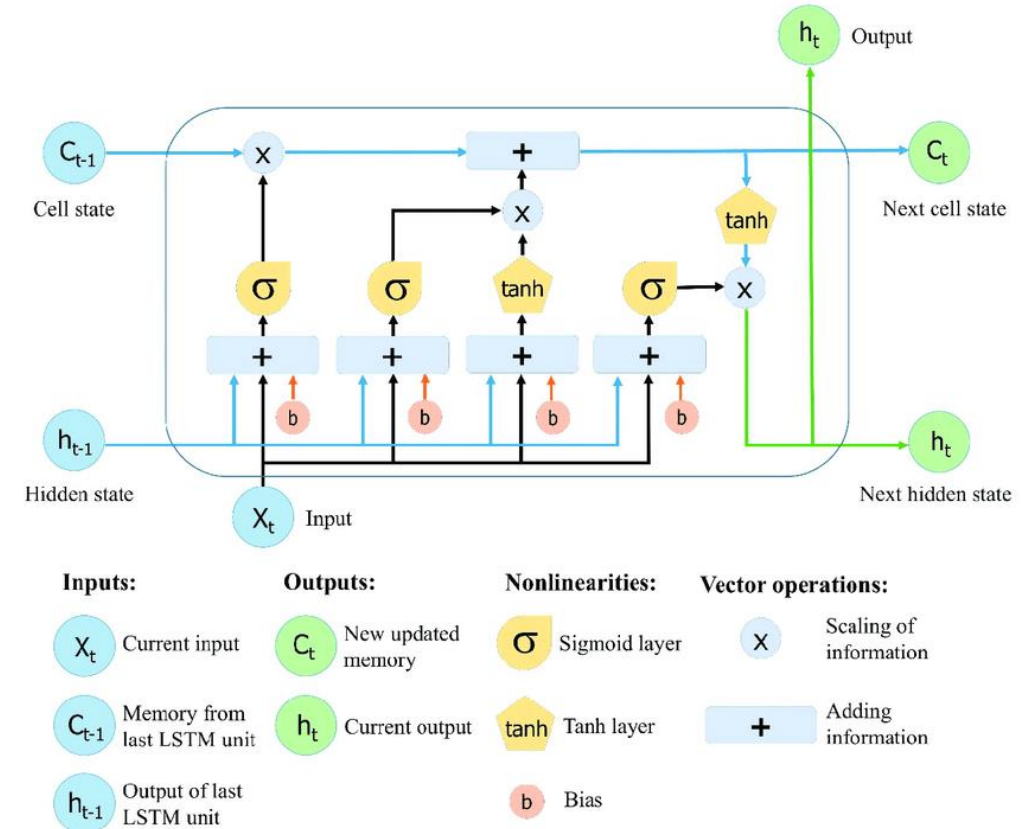
NLP Historical Context and Challenges

NLP has *long* relied on recurrent neural networks (RNNs), including LSTM and GRU architectures, which model the sequential dependencies of words to capture context.

Inherent Limitations: RNNs and their variants face significant challenges:

- **Sequential Processing:** Each state depends on the previous, inhibiting parallel processing on GPUs and increasing training times.

RNNs take $O(\text{sequence length})$ steps for distant word pairs
- **Difficulty with Long-Distance Dependencies:** RNNs struggle with capturing relationships between words that are far apart in the text, due to *vanishing gradients* and other issues related to sequence length.

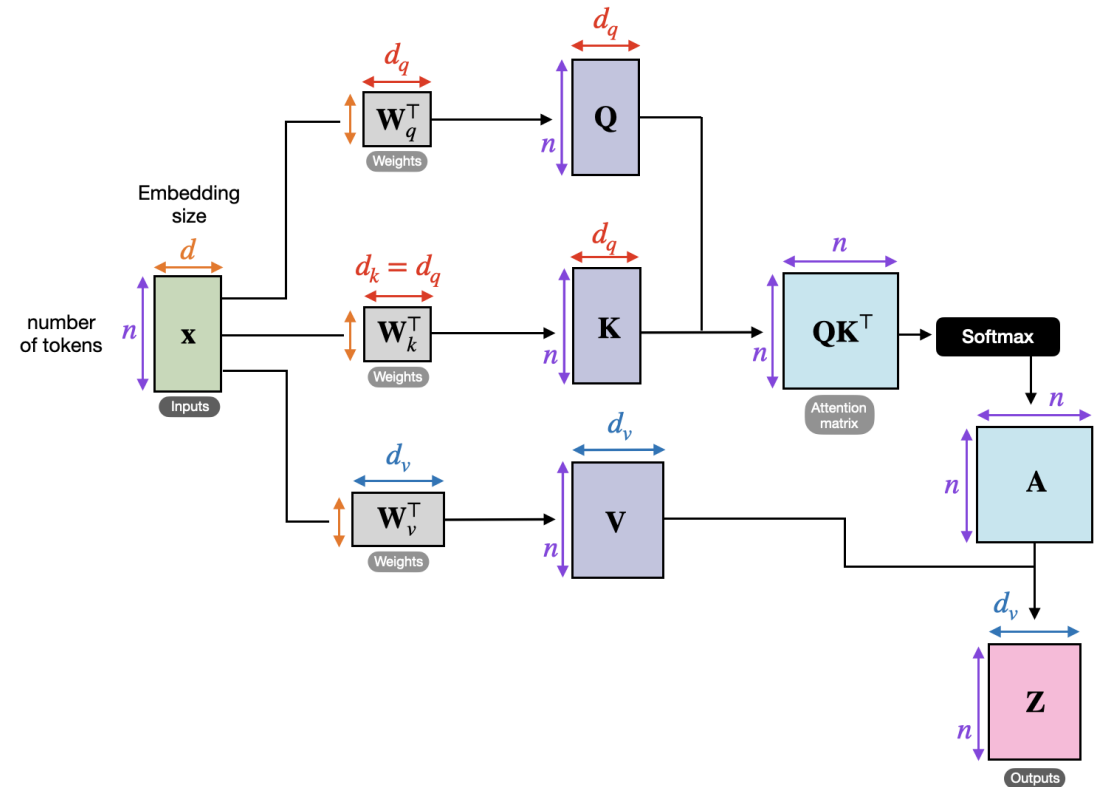


Long Short-Term Memory (LSTMs)

Graves, A. and Graves, A., 2012. Long short-term memory. *Supervised sequence labelling with recurrent neural networks*, pp.37-45.

A paradigm shift: Self-Attention

- **Attention** provides a solution to the bottleneck problem.
- Offers a **paradigm shift** by enabling models to focus dynamically on different parts of the sequence without the sequential processing bottleneck.
- Unlike RNNs, $O(1)$ complexity for interactions between any two words in the sequence.
- **Parallelizable:** Fully compatible with GPU architectures, allowing simultaneous processing of sequence data.
- **A very simple operation:** re-represents each word as a convex combination of its neighborhood



Self-Attention

Vaswani, A., 2017. Attention is all you need. *Advances in Neural Information Processing Systems*.

Self-Attention: A convex combination

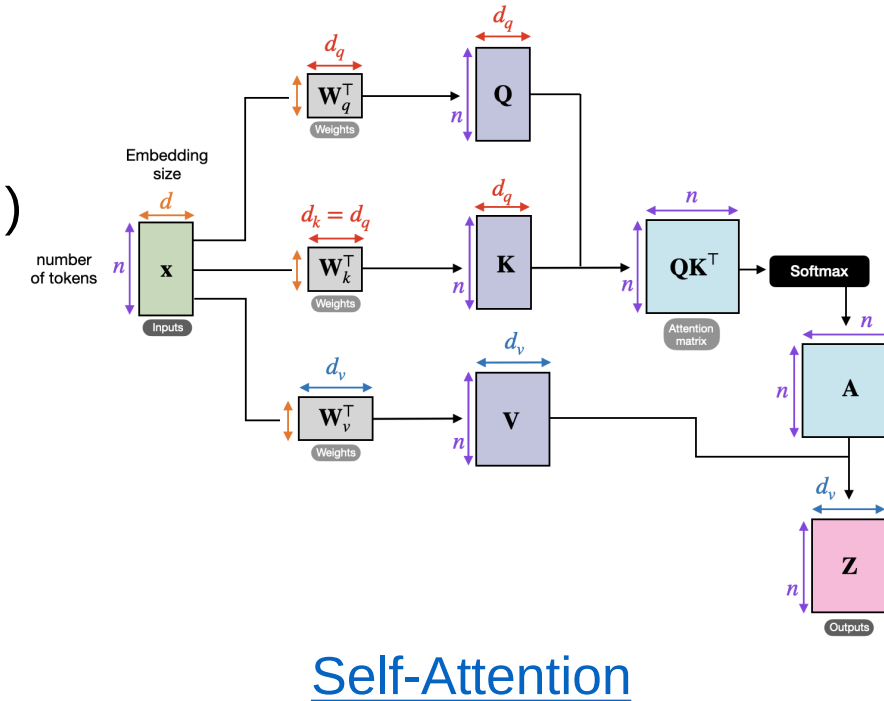
- Each word in the input sequence generates a Query (Q), Key (K), and Value (V) representation.
- These representations are computed through learned linear transformations of the word's embedding (**Recall Word Vectors!**)

$$Q = W^Q x, K = W^K x, V = W^V x$$

where x is the input embedding, W^Q , W^K , W^V are the weights matrices for Query, Key, and Value, respectively.

$$Attention(K, Q, V) = softmax\left(\frac{QK^T}{\sqrt{d^k}}\right)V$$

- Each word's output is a weighted sum of Value vectors, where the weights are determined by the relevance (*softmax of dot products*) of corresponding Keys to the Query.



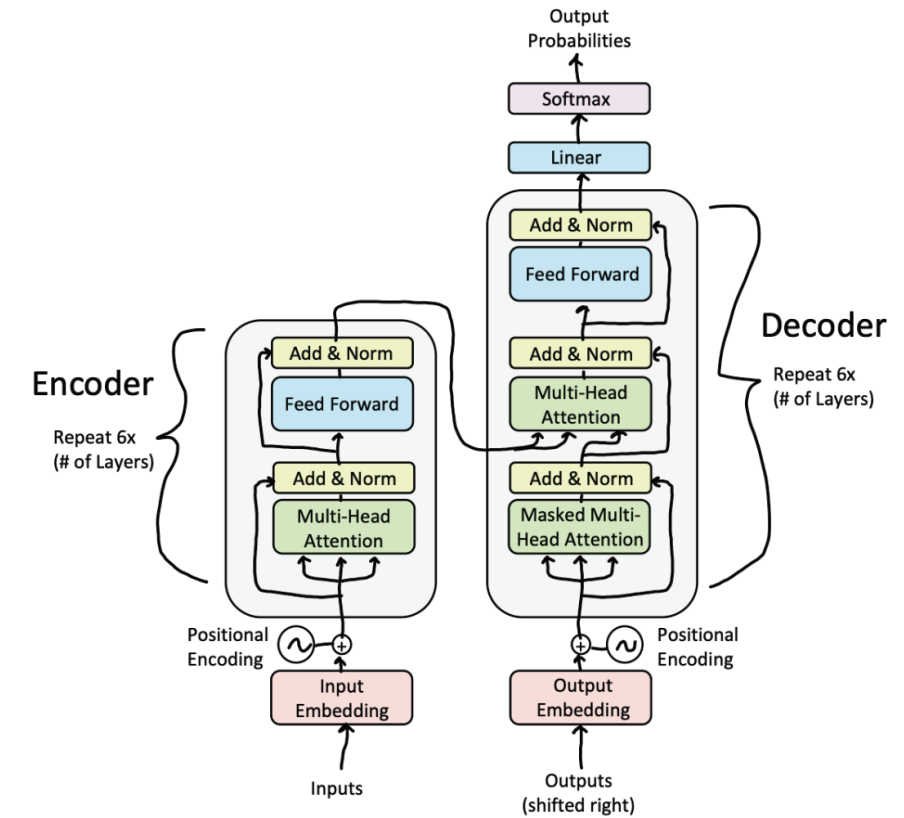
Vaswani, A., 2017. Attention is all you need. *Advances in Neural Information Processing Systems*.

Transformers: is Attention All We Need?

- The **Transformer model** was introduced in the seminal paper “*Attention is All You Need*” as a new architecture that removes recurrence entirely.
- **Self-Attention** is the core building block of Transformer.
- **Self-supervised learning** uses the words generated so far to generate next word.

Encoder

- Begins with input embeddings combined with **positional encodings** to retain sequence order information as attention is permutation invariant!
- Attention is a linear operation, and **non-linearity** is an important component to learn complex downstream language tasks.
- Therefore, encoder introduces **Feedforward** and **normalization** layers.



Vaswani, A., 2017. Attention is all you need. *Advances in Neural Information Processing Systems*.

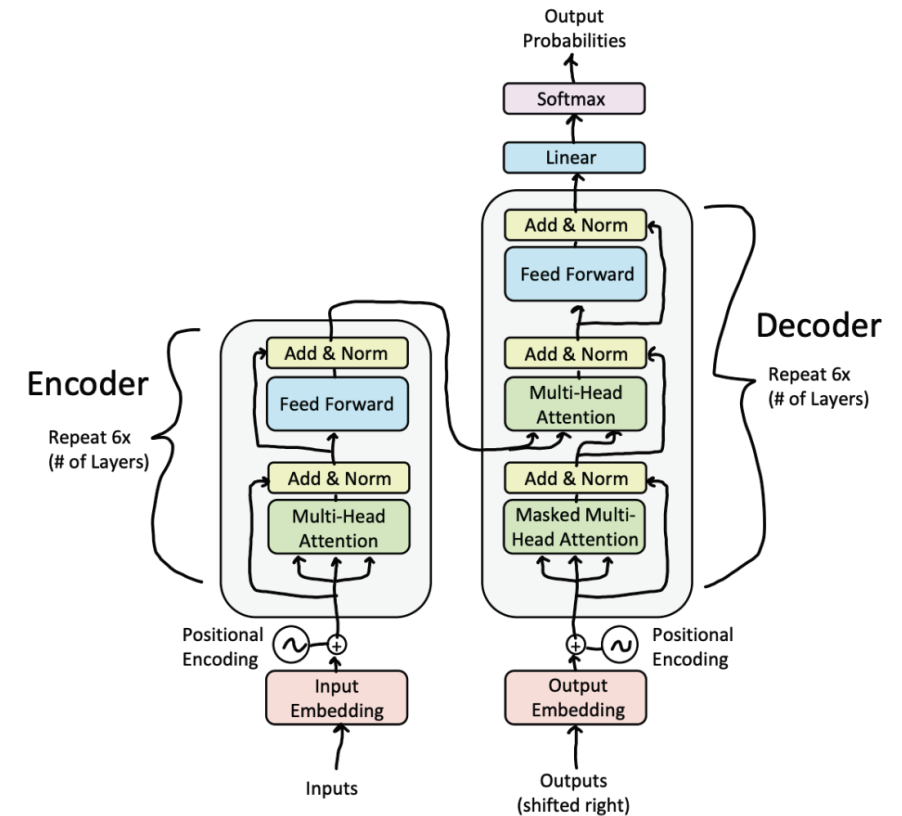
Transformers: is Attention All We Need?

Decoder

- Neural networks are very good at cheating (by accessing future words during training). Hence, decoder introduces **Masked Attention** to hide future words ([Recall Autoregressive Property!](#))
- Uses **Cross attention** to transfer context from encoder to the decoder. This is useful for sequence-to-sequence learning tasks, e.g., translating from “English” to “Spanish”

Many early variants of this architecture (BERT, ALBERT, XLNet, etc.)

Most of these variants uses “Sentence Entailment” or “Masked Language Modeling” as their learning Objective



Transformers: Beyond the field of NLP

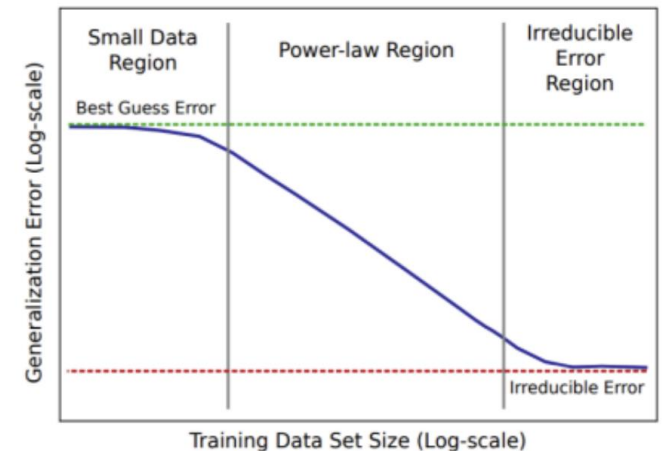
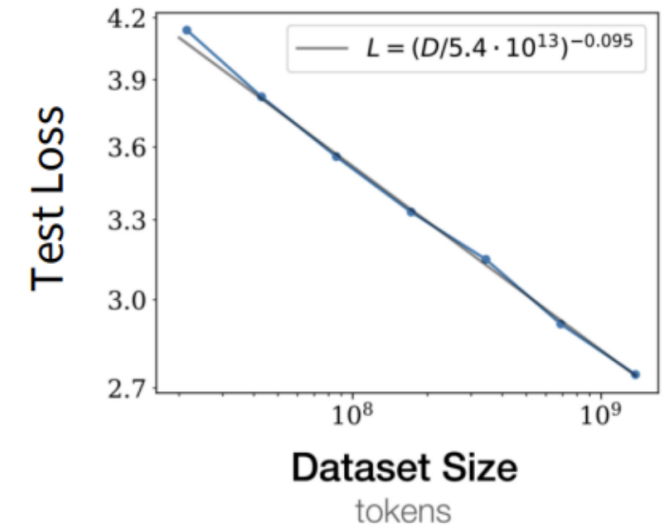
- **Attention** has proven to be a paradigm shift not just in the field of Natural language processing but in the whole field of deep learning.
- **Vision Transformers** (ViTs) apply Attention mechanism to image patches and show improvements over the existing architectures relying on Convolutional Neural Networks ([Paper](#))
- Beyond vision, attention mechanisms have shown promising results in a variety of areas, such as audio processing (speech recognition and music generation), system biology (complex biological interactions), and drug discovery (prediction of molecular interactions and properties).

Relevant Important resources to learn the architecture in detail>

- *“Attention Is All You Need”* by Vaswani et al: [Read the paper](#)
- Ashish Vaswani’s Talk : [Transformers & Attention](#)
- Yannic Kilcher: [Detailed explanation](#)

Large Language Models (LLMs)

- LLMs can be used for **different tasks** (translation, summarization, or sentiment analysis) by pre-training a “base” model and then fine-tuning for the task(s) of interest.
- This is “**power-law scaling**”: more data, better performance.
- We want models that can perform NLP tasks in a **zero-shot** manner (pre-training, fine-tuning, and prompting).
- Language models that have many parameters and can perform **multiple tasks** through **prompting**.
- E.g., GPT, Llama3, Gemini, PaLM, Claude, Mistral, Mixtral, etc.



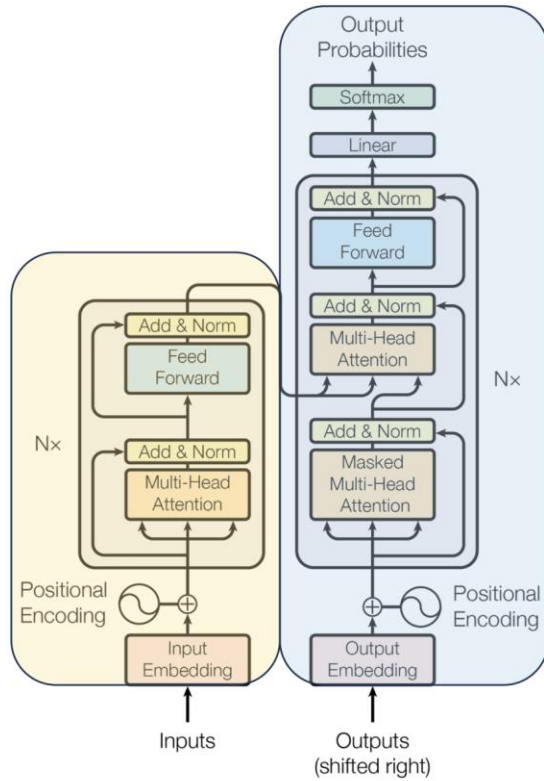
LLMs are good at a range of downstream tasks



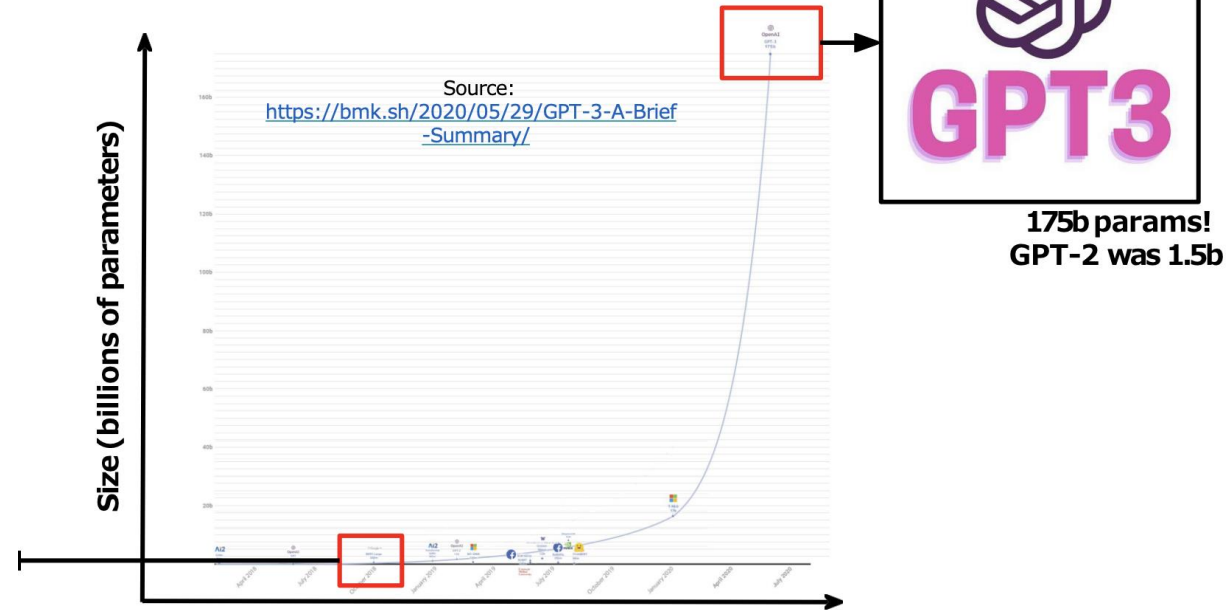
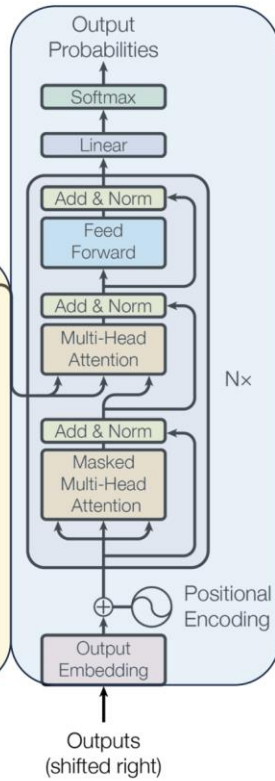
How do LLMs work?

The “LLM Era” – Paradigm Shift in ML

BERT
Oct 2018
Representation



GPT
Jun 2018
Generation



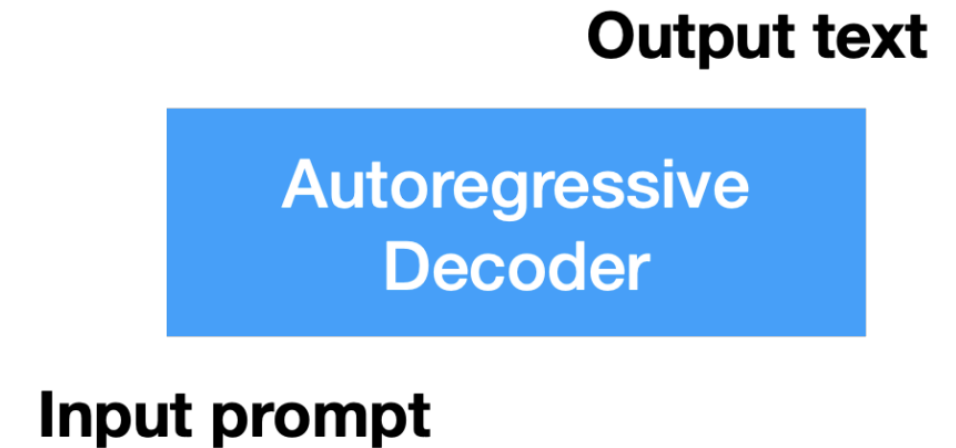
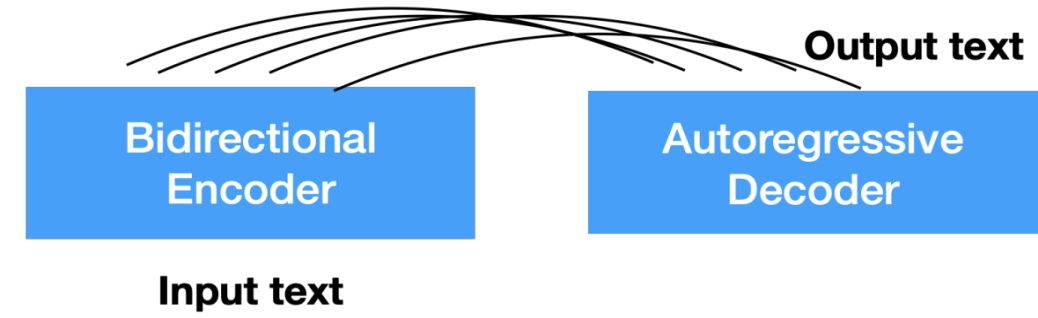
How do LLMs generate text?

- LLMs generate text by predicting the next word in a sequence based on the previous words, one word at a time, using the probability distribution learned from training data.

Ref: They walked to the grocery store .

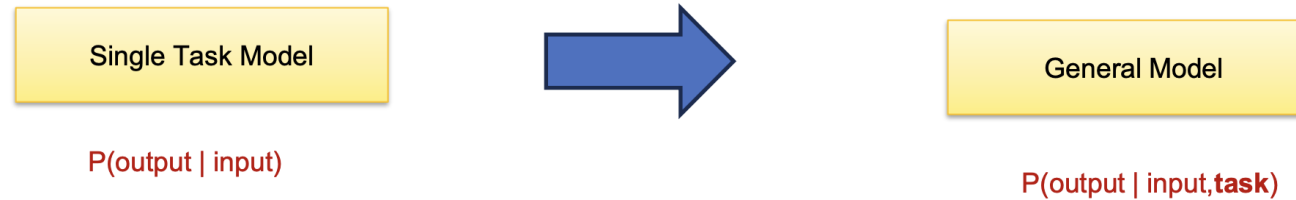
Gen: The woman went to the hardware store .

- Compute a score that indicates the **lexical similarity** between generated and gold standard (human-written) text.



Pre-prompting paradigm

- LLMs can be used for different tasks by pre-training a “base” model and then fine-tuning for the task(s) of interest

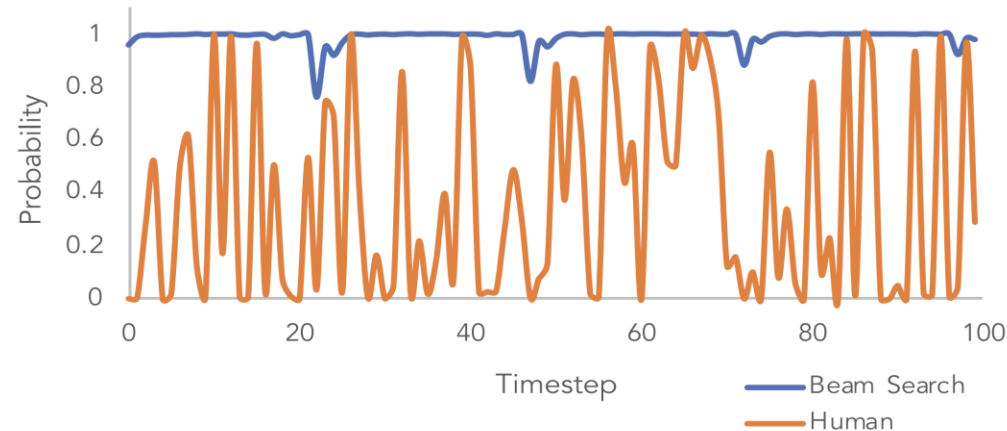


Practical Issues

- Too many copies of the model
- Need for large-scale labeled data for fine-tuning
- Can do only specific task

Why is coherent text generation so hard?

- AI relies on probability distributions, while human text often defies these distributions
- Human writing incorporates nuances, context, and creativity that are difficult for AI to replicate.
- There is no inherent pattern for AI to learn.



Recent Large Language Models

GPT by OpenAI

- We have briefly introduced Transformers before, and all the recent LLMs are variants of the same architecture.
- *Generative Pre-trained Transformer* (GPT) is a type of large language model developed by OpenAI with various variants (GPT-3.5, GPT-4, GPT-4o etc.)
- Newer versions of GPT are **not open-source**.

Pre-training and Fine-tuning

- The model is first trained on a vast corpus of text data, learning patterns, grammar, facts, and some reasoning abilities from that text (probably the whole internet!)
- It helps the model learn the structure and semantics of the language itself.
- After pre-training, GPT models are further trained on specific downstream datasets or tasks to specialize in generating text in a controlled and more useful way.

Relevant resources

- To get free access to OpenAI API: <https://platform.openai.com/playground>
- [langchain](#) to create LLMs in few lines of code.

Mistral 8x7B

- A **Sparse Mixture of Experts (SMoE)** LLM with 8 experts sub-networks summing to 45 billion parameters. Only two experts are activated per token, resulting in the compute cost of a 14B model, making it a resource-efficient choice for complex NLP tasks.
- Like the idea of *ensembles* in machine learning (Recall Random Forests) that combines the outputs of multiple models to improve accuracy, [Mixtral](#) uses **Mixture of Experts (MoE)** as specialized sub-networks (experts).
- Unlike traditional ensembles that activate all models at once, MoE dynamically selects a few experts which reduces computational costs but also enhances the model's ability to focus on the most relevant expertise for any given input, leading to improved efficiency and performance.
- Has multiple variants for different domains:
 - [BioMixtral](#) - specialized for medical texts, fine-tuned for tasks related to healthcare and biomedical applications
 - [MathΣtral](#) - Optimized for mathematical problem-solving
 - [Codestral](#) - Designed for programming tasks like code generation

LLaMA by Meta

- Built on the foundation of the Transformer architecture, LLaMA (Large Language Model Meta AI) introduces key optimizations like [Grouped Multi-Query Attention](#) and [RMS Normalization](#). (more effective handling of queries while maintaining the integrity of the data being processed, leading to better performance in various language tasks)
- LLaMA employs a [Key-Value Cache](#) to optimize next-token prediction, reducing redundant computations and speeding up inference (Linear vs [quadratic time](#) for standard attention!)
- In the last few months, Meta has released multiple open-source variants of LLaMA:
 - [LLaMA 3.2](#) – Combines vision and language processing, enabling tasks like image captioning and visual reasoning
 - [LLaMA 3.1](#) – Three variants in terms of model size (8B, 70B and 405B parameters) with capabilities in general knowledge queries, steerability (adapt its responses based on user input), math, tool usage, and multilingual translation

Conclusion

- Evolution of NLP from word embeddings to self-attention and transformers.
- LLMs: introduction and how these work and recent examples of LLMs (e.g., GPT by OpenAI, Mistral 8x7B, LLaMA by Meta).
- LLMs are transforming interactions with technology and opening new avenues in healthcare, education, and creative industries.
- Ethical Considerations: remaining mindful of ethical considerations and responsibilities in deploying these powerful models.
- LLMs as Agents in RL Environment and Search.
- Tutorial on Prompt Engineering for LLMs.

Questions?